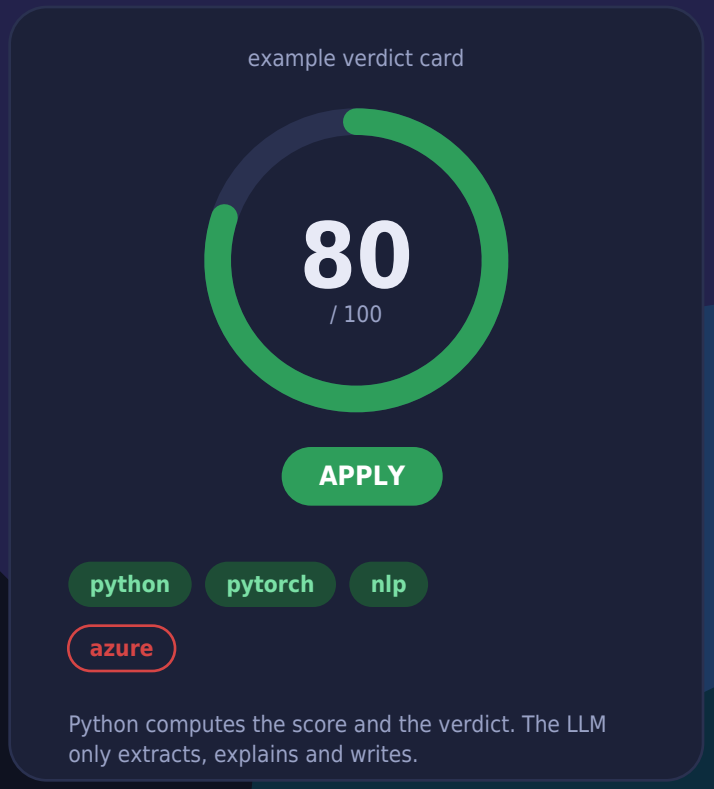
JOB SEEKERS' BROTHER

JSBrother

A multi-agent AI assistant that scores your fit, tells you APPLY, CAUTION or SKIP, then writes the application and sends it only after your approval.

Job Seeker mode Freelancer mode

APPLY CAUTION SKIP



Qwen2.5-7B · LangChain LCEL · FAISS + MiniLM · Streamlit · Gmail SMTP

Status: M0 to M8 done · M9 evaluation and M10 docs + video next

The idea in one picture

From a CV and a posting to a sent application, in four steps

1 INPUT

CV (PDF) plus a job posting or a project post

2 ANALYZE

Score 0 to 100 and a verdict, computed by Python

3 DRAFT

Tailored cover letter or proposal, fact-checked

4 SEND AND LOG

After your approval: email plus a tracker row

JOB SEEKER MODE

INPUT	CV (PDF) + posting text, URL optional
ANALYSIS	Match score, strengths, gaps
DRAFT	Tailored cover letter or email
ACTION	Email the application, then log it

FREELANCER MODE

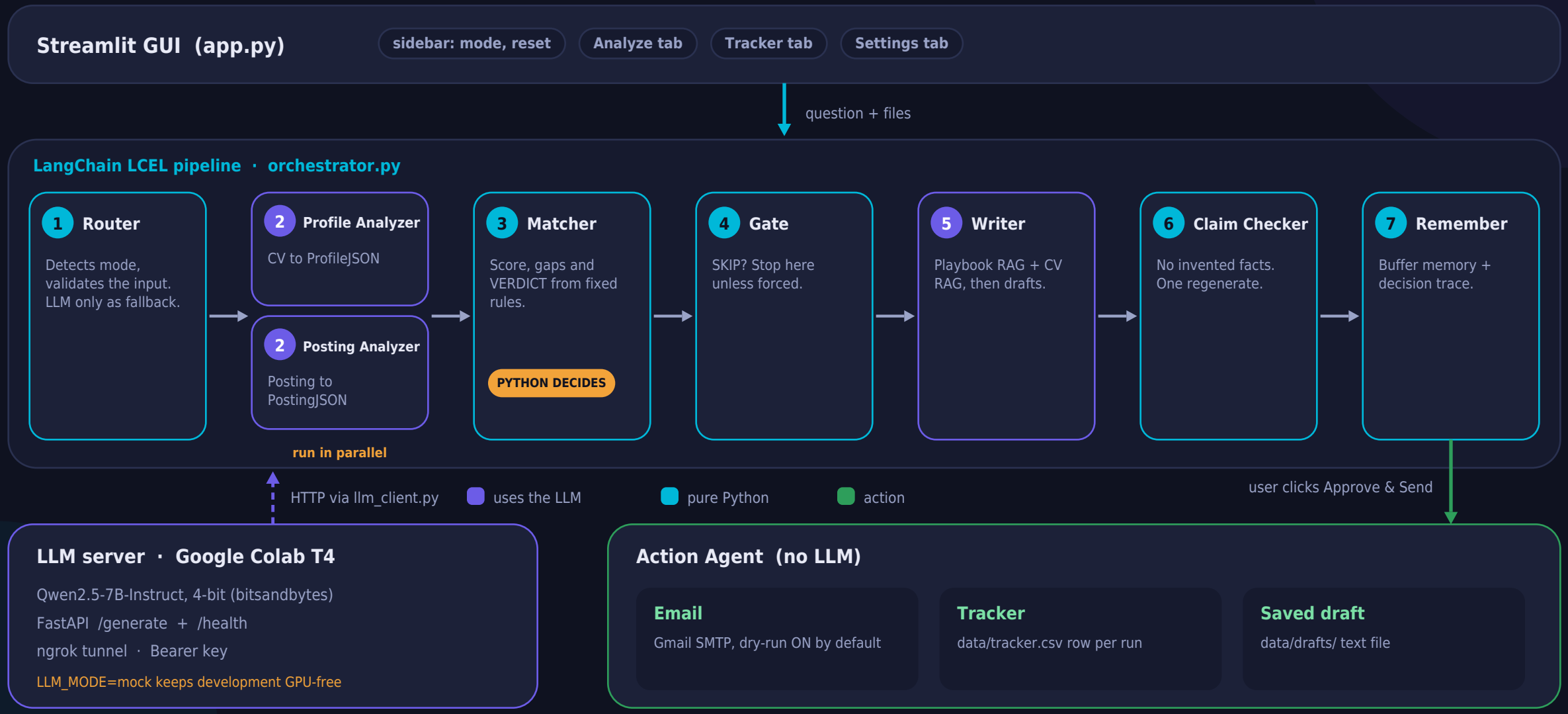
INPUT	Skills profile + project post (text)
ANALYSIS	Score, gaps, budget fit, deadline fit, red flags
DRAFT	Proposal: problem, approach, timeline, price
ACTION	Download or copy, then log (email optional)

GUI order: **1 Verdict card → 2 Editable draft → 3 Approve & Send**

On SKIP only the card shows, with an "Apply anyway" button that generates the draft.

Architecture

One GUI, one LCEL pipeline, one separate LLM server, one action agent



The six agents

Each has a role, tools and a goal. All tools are LangChain StructuredTools

1 Router

Detects the mode (job or project), checks the inputs, starts the pipeline.

TOOLS

keyword rules | short LLM fallback | LCEL runner

LLM: rarely

2 Profile Analyzer

Turns the CV into a structured ProfileJSON: skills, years, roles, projects.

TOOLS

pdf_reader (pypdf) | llm_client | json_utils (retry)

LLM: extracts

3 Posting Analyzer

Turns the posting into PostingJSON: must / nice skills, deal-breakers, budget, deadline. Retry plus a rule-based fallback if no skills come back.

TOOLS

web_fetcher (best effort) | llm_client | json_utils

LLM: extracts

4 Matcher

Computes score, gaps, verdict and advice from fixed rules.

TOOLS

skills.py aliases + MiniLM | compute_score | decide_verdict

LLM: phrasing only

5 Writer

Writes the cover letter or proposal using playbook rules and the CV's own facts.

TOOLS

playbook retriever (FAISS) | CV-chunk retriever | claim_checker

LLM: writes

6 Action Agent

Sends the approved email and logs everything.

TOOLS

email_sender (smtplib) | tracker (CSV) | save_draft

LLM: none

The verdict engine

Everything here is plain Python. The LLM never touches the score or the verdict

SCORE = weighted coverage

Job Seeker



Freelancer



- exp_fit = min(1, your years / required years), or 1 when no years are required.
- budget_fit: 1 if budget meets your minimum, 0.5 if within 20% below, 0 if lower, 0.7 if unknown.
- deadline_fit: 1 if no deadline or enough days, 0.5 if at least half, else 0.

SKILL MATCHING

normalize → exact / alias map → MiniLM cosine ≥ 0.70

Pipe alternatives such as "pytorch | tensorflow" count when any one matches.

VERDICT RULES, applied in this exact order

1	**Any deal-breaker hit** Only if your profile demonstrably fails it. Unverifiable items become warnings.	SKIP
2	**Score < 50, or 3+ missing must-haves**	SKIP
3	**Score ≥ 75 and no missing must-have**	APPLY
4	**Everything else**	CAUTION
5	**Freelancer: vague scope or budget_fit = 0** Downgrades APPLY to CAUTION and adds a red flag.	→ CAUTION

The LLM only phrases 3 reasons and 1 advice line from facts that Python already computed.



Indicative zones: deal-breakers and missing must-haves can override the score.

Writer and safety net

Drafts are grounded in your CV and checked for invented claims

Cover letter

≈ 220 to 300 words

- Hook tied to the company and role
- 2 to 3 achievements matching must-haves (CV only)
- Honest, brief handling of a gap
- Clear call to action

Proposal

≈ 150 to 250 words

- Restate the client's problem
- Approach in 3 steps
- Timeline
- Price
- One relevant proof point
- One clarifying question

In the GUI

DRAFT TYPE

Job: Cover letter · Email · Both

Freelancer: Proposal · Short bid · Both

REFINE

shorter | more formal | friendlier

or a free-text instruction

RULES

Never claim a skill from missing_must. No invented numbers, employers or years.

HOW THE WRITER GETS ITS CONTEXT (RAG)



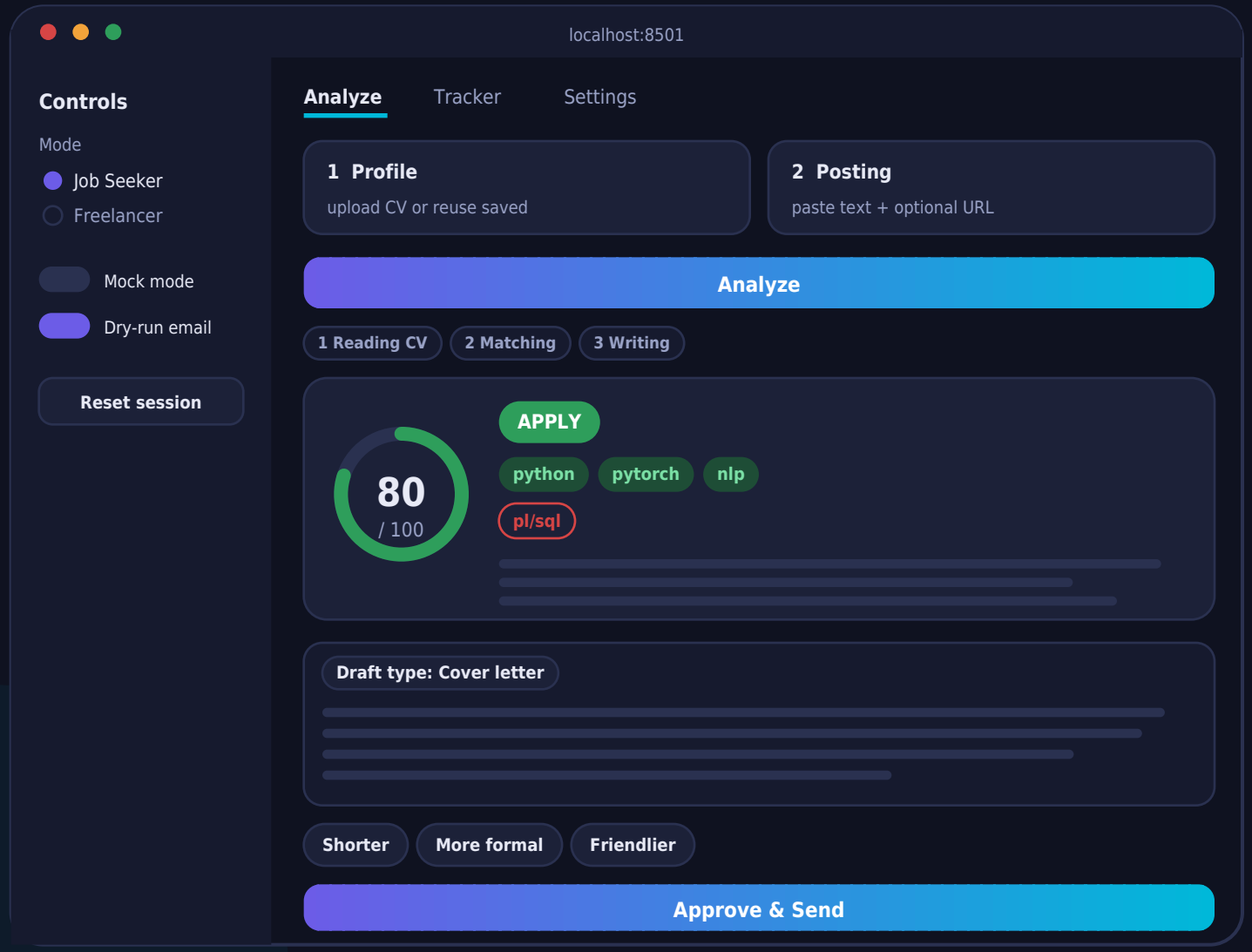
CLAIM CHECKER, the anti-hallucination loop



Catches invented numbers, years, tools and employers before the draft reaches you.

The Streamlit GUI

Midnight Aurora theme: dark, calm, with a violet to cyan gradient



THEME PALETTE

Background	Sidebar	Cards
#0F1220	#161A2E	#1C2138
Primary	**Cyan glow**	**Text**
#6C5CE7	#00B8D9	#E8EAF6
APPLY	**CAUTION**	**SKIP**
#2E9E5B	#F2A33A	#D64545

Header and buttons

#6C5CE7 → #00B8D9

Job Seeker badge | Freelancer badge

UX RULES

- Spinner plus step-by-step progress
- Friendly errors: endpoint down, bad PDF, empty posting
- Nothing is emailed without the Approve click
- Dry-run is ON by default

Data, memory and stack

Four JSON objects flow through the system; every LLM answer is validated

ProfileJSON

name, email, headline · skills[] · years_experience · roles[] · education · projects · min_budget · min_lead_days

PostingJSON

title, company_or_client · type job|project · must_have[] · nice_to_have[] · min_years · deal_breakers[] · budget · deadline_days · vague_scope

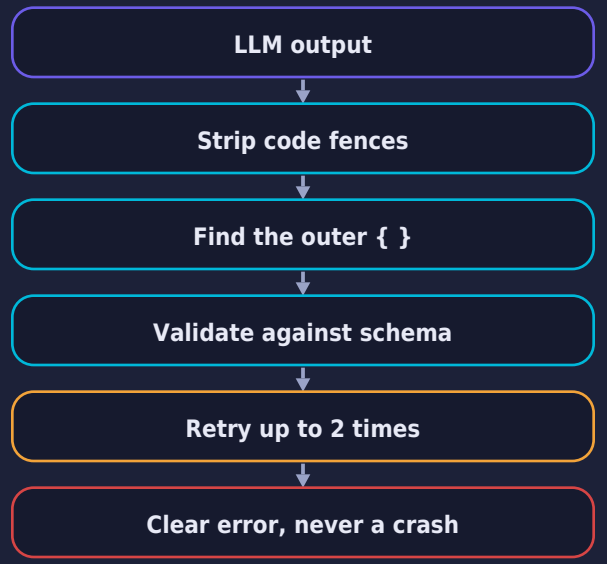
MatchResult

score 0 to 100 · must/nice coverage · matched[] · missing_must[] · deal_breaker_hits[] · verdict · reasons[3] · advice · freelancer{budget_fit, deadline_fit, red_flags}

Draft

kind cover_letter|proposal · subject · body · claims_ok · flagged_claims[]

JSON safety net



TECH STACK

LLM Qwen2.5-7B-Instruct 4-bit · bitsandbytes · Colab T4

SERVING FastAPI · ngrok

ORCHESTRATION LangChain LCEL

RETRIEVAL MiniLM · FAISS

GUI Streamlit

ACTIONS Gmail SMTP · CSV tracker

INPUT pypdf · BeautifulSoup

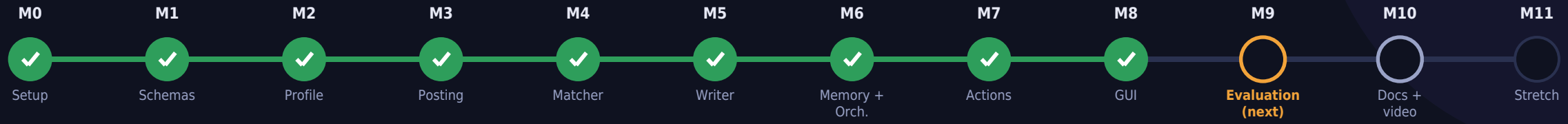
MEMORY

Buffer memory keeps the session turns · profile.json is saved so you upload the CV once · last posting and last draft feed the Refine buttons

st.session_state holds everything across Streamlit reruns

Where the project stands

M0 to M8 are built and tested; evaluation and the video come next



96.6% · 100% · 100%

CORE skill recall on 3 CVs (target ≥ 80%)

6 / 6

synthetic postings with must-have recall ≥ 80%

0 flagged

invented claims across 5 evaluated drafts

59+ tests

automated tests passing at last full run

1 real email

sent through Gmail SMTP, plus dry-run logging

11 postings

scored: 6 synthetic + 5 real LinkedIn / Freelancer

SAMPLE VERDICTS (M4 evaluation)

real_job_1 APPLY 80.0 | real_project_1 CAUTION 95.0 | real_project_2 CAUTION 82.5 | real_job_3 SKIP 45.6

WHAT IS LEFT

- M9 evaluation set + results table
- test 12 more samples → README
- upload to GitHub (.gitignore for .env)
- M10 README + face-cam video
- optional: ingest the 50-slide playbook deck

Evaluation, course mapping and video

What gets measured, what each week contributed, and the 5-minute demo plan

EVALUATION TARGETS

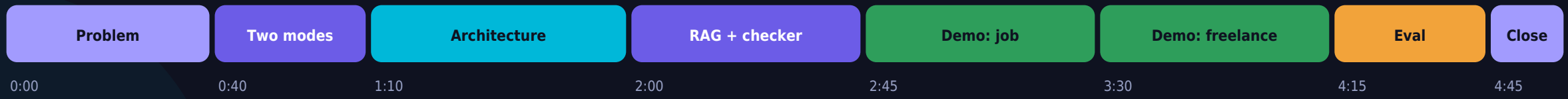
Extraction accuracy	≥ 80%	✓ 96.6 to 100% CORE
Verdict accuracy	≥ 85%	M9
Score sanity	≥ 85%	M9
Hallucination	0 flagged	✓ 0 in 5 drafts
JSON validity	first try + retry	M9
Action success	100%	✓ dry-run + real email
Efficiency	mean sec / run	M9 (one run ≈ 106 s)
Reliability	≥ 95% no crash	M9

COURSE MAPPING

1. **Week 1 · LLMs & Agents**
   Qwen model, six agents
2. **Week 2 · RAG & Fine-tuning**
   FAISS over playbook + CV
3. **Week 3 · LangChain & Memory**
   LCEL, tools, buffer memory
4. **Week 4 · Multi-agent & Automation**
   router, parallel, email + tracker
5. **Week 5 · GUI**
   Streamlit, Midnight Aurora

Objectives covered: retrieval · summarization · actions · orchestration · presentation · testing

5-MINUTE VIDEO PLAN



Tip: use runs that end in APPLY or CAUTION for the demos, because SKIP shows only the card (no draft).

Run it and avoid the traps

Start order every session, the .env keys, and the known pitfalls

START THE APP

1	**Colab**	Connect a T4 GPU, run the cells in order, server cell last. Copy the public URL.
2	**.env**	If the URL changed, paste it into LLM_BASE_URL.
3	**Check**	`.venv\Scripts\python.exe verify_remote.py`
4	**Run**	`.venv\Scripts\streamlit.exe run app.py`
5	**Open**	http://localhost:8501 (tick Mock mode to test without the LLM)

.env KEYS

```
LLM_MODE  (mock | remote)
LLM_BASE_URL  (the ngrok URL)
LLM_API_KEY  (same as the Colab secret)
SMTP_USER  (Gmail address)
SMTP_APP_PASSWORD  (Google app password)
```

Never commit .env

CV data stays local; traces never log full CV text.

KNOWN PITFALLS

ngrok URL changes
after every restart: update .env

Colab times out
use LLM_MODE=mock for GUI work

JSON drift
temp 0 + retries + validation

Writer temperature
0.4 to 0.7; extraction stays near 0

Scraping blocked
LinkedIn / Upwork: paste the text

Gmail
needs 2FA + an app password

Streamlit reruns
keep state in session_state

Long CVs / postings
chunk first, then summarize